DuckDB from the Command Line

macOS: install DuckDB and run your first queries

OMIS 105 - Fall 2026

Examples were made with DuckDB 1.5.5 and re-checked on **DuckDB 2.0.0** (2026-11-13). DuckDB **2.0** or newer is required for this course; the version banner and some messages look slightly different from the examples.

Reviewed October 2, 2026. Start here, then follow the slides in order.

1. What You Will Learn

1. Install the standalone DuckDB command-line program.
2. Fix “command not found” or “not recognized.”
3. Open the tutorial folder and query its CSV file.
4. Save a database, change output format, and run a SQL file.
5. Diagnose file, permission, and session errors.

Keep this guide and `data/students.csv` together. You will create `report.sql` during the lesson.

Follow the main lesson first. Use **Help Only If Needed** at the end only when a step fails. You do not have to memorize troubleshooting.

2. Four Words Used in This Guide

Word	Meaning here
Terminal / shell	The terminal is the window; the shell reads your commands.
CLI	“Command-line interface”: the DuckDB program used in this lesson.
Path	A file or folder's location on your computer.
PATH	The list of folders searched when you type a program name.

Enter and **Return** mean the same key in these instructions.

After each command, wait for the result before typing the next one.

3. Two Prompts, Two Kinds of Commands

Where you are	Examples to enter there
Terminal shell	`duckdb --version`, `duckdb`, `cd`
DuckDB prompt (`D`)	`SELECT 42;`, `.tables`, `.quit`

The prompt is a label: **do not type `D`, `$`, `%`, or `PS>`**.
Copy command blocks exactly. Blocks labeled “output” show results;
do not type those results.

If you see `D`, enter `.quit` before a terminal command.
If you see `>>>`, you are in Python: enter `exit()` first.

4. Before Installing

The **CLI** is the DuckDB program you run in a terminal. Having DuckDB work in Python or Marimo does not mean this program is installed.

Open **Terminal**: press Cmd+Space, type `Terminal`, and press Return.
Then run:

```
duckdb --version
```

If you see a version number such as `v2.0.0`, DuckDB is installed and your terminal can find it (if it starts with `v1.`, run `brew upgrade duckdb`: the course requires 2.0). Skip installation and go to **Open the Tutorial Folder**.
If you see “command not found” or “not recognized,” follow the installation steps next. Choose only one method. For another error, use the help section.

No Python environment, database server, or DuckDB account is needed.

5. macOS: Choose an Installation Route

Open **Terminal** with Spotlight: Cmd+Space, type Terminal, press Return.

- Already use Homebrew? Choose Option A.
- No Homebrew or no administrator access? Choose Option B: a manual download in your own home folder.
- A managed Mac may require instructor/IT approval for either route.

Use the **current stable CLI** on duckdb.org/install, not a preview, Python package, or library download. Check OS compatibility there.

6. Mac Option A: Existing Homebrew

In Terminal:

```
brew --version
brew install duckdb
duckdb --version
```

Run each command separately. If the final command, **duckdb --version**, prints a version number, skip Option B and go to **Mac: Open the Tutorial Folder**. A Homebrew version alone does not confirm DuckDB is installed.

If `brew` is missing, use **Option B**. You do not need to install Homebrew just to finish this lesson.

7. Mac Option B: Download and Extract

1. At duckdb.org/install, choose **macOS / CLI / current stable**.
2. Download **duckdb_cli-osx-universal.zip**. It supports Apple Silicon and Intel; no chip-specific CLI zip is needed.
3. Unzip it in Finder. Locate the actual file named **duckdb**.
4. In Finder, open your Home folder (Shift+Cmd+H), create a folder named **duckdb-cli**, and move the executable into it.

The next slide expects `~/duckdb-cli/duckdb`. If extraction made an extra folder, move the executable out of that folder.

8. Mac Manual Install: Test the Full Path

In Terminal:

```
ls -l "$HOME/duckdb-cli/duckdb"
chmod +x "$HOME/duckdb-cli/duckdb"
"$HOME/duckdb-cli/duckdb" --version
```

If `ls` reports “No such file,” correct the location in Finder first.
If macOS blocks the file, see **Mac: Permission or Security Errors**
in the help section at the end.

A version number means the executable works, even if typing just `duckdb`
still says “command not found.” No `sudo` is needed for this folder.

9. Mac Manual Install: Add Your Folder to PATH

PATH is the list of folders your shell searches for a command name.

For this Terminal session:

```
export PATH="$HOME/duckdb-cli:$PATH"
duckdb --version
```

To keep this setting when you open Terminal again, run once (standard zsh):

```
printf '\nexport PATH="$HOME/duckdb-cli:$PATH"\n' >> "$HOME/.zshrc"
```

Copy the whole line as shown; it saves this setting for future sessions.

`$HOME` means your home folder: leave that text unchanged. Open a new Terminal window and run `duckdb --version` again.

10. Mac: Open the Tutorial Folder

Download or clone the **whole tutorial folder**, then unzip it if needed.
Do not save a GitHub HTML page in place of `students.csv`.

In Terminal, type `cd ` (including the space), drag the actual tutorial folder from Finder into Terminal, then press Return. Next run:

```
pwd
ls data/students.csv
```

The CSV must exist. Use your actual folder location, not an assumed Desktop path. For spaces, use quotes: `cd "$HOME/Desktop/OMIS 105 Labs"`.
Leave `$HOME` as shown; do not write `"~/Desktop/..."` instead.

11. Checkpoint: Does DuckDB Run?

In your terminal:

```
duckdb --version
duckdb -c "SELECT 6 * 7 AS answer;"
```

The first command should print a version number; the second should return **42**. Example version output (do not type this):

```
v2.0.0 (Cyanoptera) d54cbbe3df
```

If typing `duckdb` still fails, see **Help Only If Needed** at the end.
It explains how to start DuckDB using the location of its program file.

12. Checkpoint: Can It Read the Sample?

In your terminal, in the tutorial folder:

```
duckdb -c "SELECT COUNT(*) AS students FROM read_csv('data/students.csv');"
```

Expect **8**. If this fails but the 42 check worked, installation is OK:
check your current folder and the CSV filename.

The CSV should begin `id,name,major,gpa`. A file containing HTML is a saved web page, not the dataset. Download the actual file or whole repo.

Once both checkpoints pass, proceed with the SQL lesson.

13. Start DuckDB and Type a Query

In your terminal:

```
duckdb
```

When you see **D**, enter this SQL and press Enter/Return:

```
SELECT 6 * 7 AS answer;
```

You should see **42**. Startup text and spacing vary by version.

This session is **in-memory**: its database tables disappear on exit.

Writing a separate file is different; exported files remain on disk.

14. Finish or Cancel an Incomplete Command

SQL statements finish with a **semicolon** (`;`). DuckDB may show a continuation prompt if a statement, quote, or parenthesis is incomplete.

- For a missing semicolon, type `;` and press Enter.
- If you pasted the wrong text or have unmatched quotes, press **Ctrl+C** to cancel the pending input, then re-enter the complete command.
- Dot-commands such as `.quit` must start on their own fresh line. They have **no semicolon**.

A continuation prompt does not mean every problem is a missing `;` .

15. Query the CSV Directly

At the DuckDB prompt, run each block separately:

```
SELECT * FROM read_csv('data/students.csv');
```

```
SELECT major, ROUND(AVG(gpa), 2) AS avg_gpa
FROM read_csv('data/students.csv')
GROUP BY major
ORDER BY avg_gpa DESC, major;
```

No import step is needed. This reads the CSV; it does not create a stored table named `students`. DuckDB looks for `data/` inside the folder from which you started the program.

16. Expected GPA Results

major	avg_gpa
Information Systems	3.83
Accounting	3.50
Finance	3.35
Marketing	3.35

Finance and Marketing tie; sorting by `major` breaks the tie consistently.

DuckDB may display `3.5` rather than `3.50`; they are the same number.

The default **duckbox** format may also show column types.

17. Create a Table for This Session

At the DuckDB prompt:

```
CREATE TABLE students AS
SELECT * FROM read_csv('data/students.csv');
```

Then enter these **one line at a time**:

```
.tables
.schema students
```

`.tables` lists stored tables/views, not every CSV in the folder.

If `students` already exists, query it instead of running CREATE again.

18. Change How Results Look

At the DuckDB prompt, one line at a time:

```
.mode csv
.headers on
SELECT id, name FROM students ORDER BY id LIMIT 2;
```

Expected CSV output:

```
id,name
1,Alice
2,Bob
```

Other modes: `.mode markdown`, `.mode json`, `.mode table`.

Use `.mode duckbox` to return to the default display.

19. Headers and Timing

At the DuckDB prompt, one line at a time:

```
.mode csv
.headers off
SELECT COUNT(*) FROM students;
.headers on
.timer on
SELECT COUNT(*) FROM students;
.timer off
.mode duckbox
```

The first count is **8** without a header. The second adds a header and timing information. Timing values vary; these settings persist in the session.

20. Save a Database and Reopen It

First exit the in-memory session with `.quit`. **In your terminal:**

```
duckdb class_cli.db
```

At D, create the table once in this file database:

```
CREATE TABLE students AS
SELECT * FROM read_csv('data/students.csv');
```

Enter `.quit`, then run `duckdb class_cli.db` again in the same folder.
At D, `SELECT COUNT(*) FROM students;` should still return **8**.
If the table already exists, skip CREATE and query it.

21. Keep Track of Your Saved Database

- A filename such as `class_cli.db` refers to a file in the current folder. Starting from a different folder can create a different, empty database.
- Use `.databases` at D to inspect the open database; use `.tables` to inspect its tables. Quote terminal paths containing spaces.
- A database's parent folder must already exist and be writable.
- `.open file.db` switches databases; it does **not** save/copy your previous in-memory tables into the file.
- The statements in this lesson save automatically in a file database. If your instructor has you use `BEGIN`, use `COMMIT` before exiting.

22. Create a Script File

In a plain-text editor, save these lines as **report.sql** in the tutorial folder. Use VS Code, or choose **Format > Make Plain Text** in TextEdit. Check that the file is named `report.sql`, not `report.sql.txt`.

```
SELECT COUNT(*) AS students FROM read_csv('data/students.csv');
SELECT major, ROUND(AVG(gpa), 2) AS avg_gpa
FROM read_csv('data/students.csv')
GROUP BY major
ORDER BY avg_gpa DESC, major;
```

23. Run Your Script File

After saving **report.sql**, exit DuckDB with `.quit` first.

In the terminal, from this tutorial folder:

```
duckdb -bail -f report.sql
```

Expect the row count **8**, then the four GPA results shown earlier.

`-bail` stops if an error occurs. `-f` runs the SQL file and exits.

Alternatively, if you have not exited DuckDB, enter `.read report.sql` at D. In either case, start DuckDB from the tutorial folder so it can find `data/students.csv`.

24. Optional Practice: Write Another SQL File

Use VS Code or TextEdit in plain-text mode.

Save this as **my_query.sql** in the tutorial folder:

```
SELECT major, COUNT(*) AS n
FROM read_csv('data/students.csv')
GROUP BY major
ORDER BY major;
```

Exit DuckDB first, then run `duckdb -bail -f my_query.sql` in the shell.

Check it is not `my_query.sql.txt` or rich text. In TextEdit, choose **Format > Make Plain Text**. Use straight quotes, not smart/curly quotes.

25. Quotes, Spaces, and Copy/Paste

For simple one-off SQL, double-quote the shell argument and single-quote SQL strings:

```
duckdb -c "SELECT 'Finance' AS major;"
```

SQL uses single quotes for text values; double quotes are for names, such as column names.
The terminal may treat `$`, `!`, and some quotes as special instructions.
For SQL containing these characters or multiple lines, use a `.sql` file.

Copy commands from Markdown when possible. PDF line wraps, curly quotes, and copied prompt labels can alter a command. Enter one block at a time.

26. Practice: Seven Checks

1. In the shell: run `duckdb --version` and the 42 checkpoint.
   Expect a version number and the answer **42**.
2. Confirm you are in this tutorial's folder.
3. In the shell: run the CSV checkpoint and confirm the count is **8**.
4. Run `duckdb` ; at D, run the average-GPA query.
5. Try `.mode markdown` , then rerun the query.
6. Create `report.sql` as shown earlier. Enter `.quit` , then run `duckdb -bail -f report.sql` in the shell.
7. Create/reopen `class_cli.db` and verify its table persists.

If typing `duckdb` fails, use **Help Only If Needed** below.
First get the program working, then check the location of your data.

27. Help Only If Needed

If you completed all seven checks and saw the expected results, you have finished the lesson.
Use the following pages only for the problem you are seeing:

- `duckdb` is not found or not recognized.
- The program exists but cannot open.
- A CSV, SQL file, or saved table cannot be found.
- The database is locked, or the display looks unexpected.

Try the matching fix. If it does not help, use **Still Stuck?** at the end to send the instructor the details needed to help you.

28. Mac: “duckdb: command not found”

This means the shell cannot find the executable by name.
It does **not** necessarily mean DuckDB is broken.

1. If installation failed, resolve that error or use the manual route.
2. Test the executable by full path (next slide).
3. If that works, fix PATH for the route you installed.
4. Restart Terminal; for VS Code, quit and reopen the whole application.
5. Verify with `command -v duckdb` and `duckdb --version`.

Reopening Terminal alone cannot fix a missing executable or PATH entry.

29. Mac: Locate the Executable

In Terminal, try the path for your installation:

```
"$HOME/duckdb-cli/duckdb" --version
/opt/homebrew/bin/duckdb --version
/usr/local/bin/duckdb --version
```

These are alternatives, not three required installations.

The last two are usual Apple Silicon and Intel Homebrew locations.

If you previously used DuckDB's installer script, also check `"$HOME/.duckdb/cli/latest/duckdb" --version`.

30. Mac: Repair Homebrew PATH

If `/opt/homebrew/bin/brew` exists, use:

```
eval "$(/opt/homebrew/bin/brew shellenv)"
brew list duckdb
duckdb --version
```

For Intel Homebrew, substitute `/usr/local/bin/brew` in the first line.

If DuckDB is not installed, run `brew install duckdb`.

Follow Homebrew's printed startup-file instructions for a lasting fix.

Use `type -a duckdb` to find multiple commands or aliases with that name.

A working full path lets you continue while resolving PATH.

31. Mac: Permission or Security Errors

- **Permission denied:** confirm you selected the executable, then use `chmod +x` on that file. Choose a folder you own.
- **Apple cannot verify the developer:** verify the download source; follow **System Settings > Privacy & Security > Open Anyway** if macOS offers it and your device policy permits it.
- **Damaged / malware warning / policy restriction:** do not disable system protection. Ask the instructor or IT before proceeding.
- **Bad CPU type / unsupported OS:** get the current macOS universal CLI and check supported OS requirements; avoid Linux binaries.

See Apple's instructions. Homebrew does not bypass device policy.

32. Mac: Continue Without Fixing PATH First

If the manual-install program runs by full path, you can use it for SQL:

```
"$HOME/duckdb-cli/duckdb" -c "SELECT 42;"
```

To start an interactive session:

```
"$HOME/duckdb-cli/duckdb"
```

Replace the initial `duckdb` in other terminal commands with that quoted path. Use your actual installation location if different.
This avoids name-lookup problems; it does not bypass security restrictions.

33. Recovery: CSV or SQL File Not Found

1. If you are at D, press Ctrl+C only if input is unfinished; then enter `.quit`.
2. In the shell, check the current folder and list the actual filename.
3. Change into the folder containing **data/**. For scripts, also check that you created **report.sql** there as instructed.
4. Retry the eight-row checkpoint. If your problem was with the script, also retry `duckdb –bail –f report.sql` after confirming the file exists.

If using a synced folder (OneDrive/iCloud), download files locally first.
A full path can avoid ambiguity; use single quotes for paths in SQL.
If a path contains an apostrophe, double it inside the SQL string.

A `.csv` extension alone does not prove the file is valid CSV.

34. Recovery: Table, Lock, or Write Errors

- **Table does not exist:** you may be in a new in-memory session or the wrong database. Query the CSV directly or reopen the right `.db`.
- **Table already exists:** reuse it; do not drop it just to clear the error.
- **Could not set lock:** close other CLI sessions or Marimo/Python connections using that file, then retry. Do not delete lock/WAL files.
- **Read-only / cannot open:** check the parent folder, permissions, disk space, and whether you opened with `-readonly`.

Prefer a local writable folder for the database. Ask the instructor about version incompatibility errors; do not overwrite the existing database.

35. Recovery: Display or Startup Problems

- Garbled table borders? At D, try `.mode table` for ASCII borders. `.mode ascii` means a different machine-oriented format.
- Results look different? Check `.show`; reset with `.mode duckbox` and `.headers on`. A startup `.duckdbrc` can customize settings.
- To bypass it, first save a plain-text **clean_start.sql** containing only `-- No startup settings`. Save it in the tutorial folder, exit DuckDB with `.quit`, and run this command from that folder:

```
duckdb -init clean_start.sql -c "SELECT 42 AS answer;"
```

Do not delete personal startup files. Different colors or layouts do not change query results.

36. Other File and Download Problems

- SQL files with spaces in their names need quotes. In the terminal: `duckdb -bail -f "my reports/report.sql"`.
- A failed download may be caused by a school network, proxy, or certificate restriction. Ask IT; do not disable certificate checks.
- If the computer or operating system is unsupported, ask the instructor for an approved alternative computer.
- If you have not created `report.sql` or `clean_start.sql`, follow the earlier instructions before trying to run either file. They are not supplied.

37. Still Stuck? Send Useful Details

Send the instructor:

- Your macOS version and terminal application (for example, Terminal).
- The exact command and complete error text.
- Whether a full-path `--version` command works.
- Your current folder and whether `data/students.csv` exists.
- `duckdb --version`, if available; redact private path information.

Use the help pages in this guide and the official references below.
Examples were made with 1.5.5 and re-checked on 2.0.0; check `.help` and `duckdb -help` for your installed version.

38. Official References and Validation

- DuckDB installation
- DuckDB CLI overview
- CLI arguments
- Dot-commands
- Output formats
- Homebrew installation and PATH setup
- Apple: opening downloaded applications safely

SQL, CSV, scripts, saved databases, and full-path commands were tested on macOS with DuckDB 1.5.5 and re-checked with DuckDB 2.0.0. Installation permissions depend on your Mac.